



MEMORIA

TFG

Unai Pastor Martínez

ÍNDICE

Índice

Resumen..... 3

Summary 4

1. Memoria del proyecto..... 5

 1.1 Justificación del proyecto 5

 1.2 Objetivos del proyecto 5

2. Aspectos teóricos 6

 2.1.1 ¿Qué es una SPA (Single Page Application)? 6

 2.1.2 Roles de usuario en Triptoyou 6

2.2 ¿Qué es Angular? 7

 2.2.1 Uso de Angular en Triptoyou 7

2.3 ¿Qué es Laravel? 7

 2.3.1 Uso de Laravel en Triptoyou 8

2.4 ¿Qué es PostgreSQL y Neon.tech? 8

 2.4.1 Uso de PostgreSQL en Triptoyou..... 8

2.5 ¿Qué es la Inteligencia Artificial Generativa (Google Gemini)? 9

 2.5.1 Uso de Gemini en Triptoyou 9

2.6 ¿Qué es DDEV y Docker? 9

 2.6.1 Uso de DDEV en Triptoyou 10

2.7 ¿Qué es vercel? 10

2.8 ¿Qué es Railway? 10

2.9 Estado del arte 10

3. Planificación 11

 3.1 Riesgos posibles 11

 3.2 Actuaciones previstas ante los riesgos..... 12

 3.3 Presupuesto 12

 3.4 Entregables..... 13

 3.5 Metodologías ágiles 13

 3.5.1 Sprint 1 13

 3.5.2 Sprint 2 14

 3.5.3 Sprint 3 14

 3.5.4 Sprint 4 15

3.5.5 Sprint 5	15
4. Análisis.....	16
4.1 Tipos de usuarios.....	16
4.2 Requisitos funcionales.....	16
4.3 Identificación de casos de uso	19
5 Diseño de la base de datos.....	19
5.1 Esquema Relacional en PostgreSQL (Neon.tech).....	20
5.2 Tablas	20
5.3 Arquitectura de despliegue	22
6. Implementación del sistema	23
6.1 Arquitectura del sistema	23
6.1.1 El Frontend (Cliente) - Angular	23
6.1.2 El Backend (Servidor) - Laravel	24
6.2 Comunicación y Gestión de Datos	25
6.2.1 Conexión a PostgreSQL (Neon.tech)	25
6.2.2 Ejemplo de Flujo de Datos (Obtener Viajes)	26
6.3 Gestión de Viajes, Reseñas y Usuarios	26
6.4 Sistema de Reservas y Confirmación Digital	26
6.5 Analíticas, Moderación y Panel de Administración.....	27
6.6 Pasarela de pago: Integración con Stripe.....	27
6.7 Ciberseguridad Avanzada y Autenticación de Doble Factor (2FA).....	27
6.8 Entorno de Desarrollo y Portabilidad (Docker y DDEV)	28
6.9 Dependencias y Librerías Externas (Ecosistema)	28
6.10 Rendimiento y Optimización Web	30
6.11 Problemas encontrados y soluciones técnicas.....	31
7. Pruebas de integración.....	32
7.1 Gestión de usuarios y Seguridad (2FA)	32
7.2 Gestión del Catálogo de Viajes.....	34
7.3 Gestión de Reservas	35
7.4 Sistema de Reseñas	36
7.5 Asistencia por Inteligencia Artificial (Chatbot Gemini)	37
8. Manual	38
9. Conclusión y ampliaciones	38
Posibles ampliaciones	38
10. Bibliografía	39

Resumen

Este proyecto, denominado **Triptoyou**, es una plataforma web integral diseñada para la gestión y reserva de viajes, desarrollada bajo una arquitectura web moderna de tipo *Single Page Application* (SPA). Utiliza un enfoque cliente-servidor para separar claramente la presentación e interfaz de usuario (desarrollada con **Angular**) de la lógica de negocio y el acceso a datos (construido como una API RESTful utilizando **Laravel**).

El sistema cuenta con dos módulos principales:

- **Un módulo de administración**, donde los gestores pueden administrar el catálogo de viajes, supervisar las reservas y gestionar a los usuarios de la plataforma de forma centralizada y segura.
- **Un módulo para clientes**, donde los usuarios pueden explorar el catálogo aplicando diversos filtros, visualizar destinos mediante mapas interactivos, realizar reservas, publicar reseñas de sus experiencias y recibir asistencia instantánea mediante un chatbot impulsado por Inteligencia Artificial.

Para garantizar la alta disponibilidad y escalabilidad de los datos, el sistema utiliza **PostgreSQL** como base de datos principal en la nube (alojada en el servicio **Neon.tech**). Esto permite que cualquier actualización en el catálogo, precios o disponibilidad se refleje instantáneamente en todos los dispositivos conectados, eliminando problemas de sincronización local.

He elegido el stack de **Angular y Laravel** por varias razones de peso:

- **Entorno robusto y escalable:** Laravel ofrece un ecosistema maduro en el backend con herramientas avanzadas para el enrutamiento, validación de datos y la interacción fluida con bases de datos.
- **Seguridad avanzada:** Se han integrado bibliotecas especializadas para la protección de la API (Sanctum) y se ha implementado un sistema de **Autenticación de Doble Factor (2FA)** mediante códigos TOTP, garantizando la protección de las cuentas de usuario frente a accesos no autorizados.
- **Innovación tecnológica:** La facilidad para consumir e integrar APIs externas ha permitido incorporar **Google Gemini**, dotando a la plataforma de un asistente virtual inteligente capaz de interactuar con el usuario y recomendar viajes basándose en el catálogo real.
- **Comunidad y soporte:** Ambas tecnologías cuentan con inmensas comunidades de desarrolladores a nivel global y documentación extensa, lo que facilita el despliegue, la resolución de problemas y la viabilidad del proyecto a largo plazo.

Este sistema busca modernizar el sector de las agencias de viajes, ofreciendo una solución sumamente accesible, segura y con un alto valor añadido en la experiencia de cliente. Para la interfaz de usuario, se ha elegido **Angular** combinado con **Tailwind CSS**. Esto permite un desarrollo altamente eficiente, un rendimiento óptimo en el navegador y un diseño completamente fluido y adaptable a cualquier dispositivo móvil o de escritorio.

Summary

This project, called Triptoyou, is a comprehensive web platform designed for travel management and booking, developed under a modern Single Page Application (SPA) architecture. It uses a client-server approach to clearly separate the presentation and user interface (developed with Angular) from the business logic and data access (built as a RESTful API using Laravel).

The system features two main modules:

- **An administration module**, where managers can administer the travel catalog, oversee bookings, and manage platform users in a centralized and secure manner.
- **A client module**, where users can explore the catalog by applying various filters, view destinations through interactive maps, make bookings, publish reviews of their experiences, and receive instant assistance through an Artificial Intelligence-powered chatbot.

To ensure high availability and scalability of data, the system uses PostgreSQL as the primary cloud database (hosted on the Neon.tech service). This allows any update in the catalog, prices, or availability to be reflected instantly across all connected devices, eliminating local synchronization issues.

I have chosen the Angular and Laravel stack for several compelling reasons:

- **Robust and scalable environment:** Laravel offers a mature backend ecosystem with advanced tools for routing, data validation, and seamless interaction with databases.
- **Advanced security:** Specialized libraries for API protection (Sanctum) have been integrated, and a Two-Factor Authentication (2FA) system using TOTP codes has been implemented, guaranteeing the protection of user accounts against unauthorized access.
- **Technological innovation:** The ease of consuming and integrating external APIs has allowed the incorporation of Google Gemini, equipping the platform with an intelligent virtual assistant capable of interacting with the user and recommending trips based on the actual catalog.
- **Community and support:** Both technologies have massive global developer communities and extensive documentation, which facilitates deployment, troubleshooting, and the long-term viability of the project.

This system seeks to modernize the travel agency sector, offering a highly accessible, secure solution with significant added value to the customer experience. For the user interface, Angular combined with Tailwind CSS was chosen. This allows for highly efficient development, optimal browser performance, and a completely fluid design adaptable to any mobile or desktop device.

1. Memoria del proyecto

1.1 Justificación del proyecto

Triptoyou es un proyecto que nace de mi gran pasión por viajar y de mi experiencia personal como usuario de diversas plataformas turísticas a lo largo de los años. Al haber explorado tantas plataformas de viajes, he podido experimentar de primera mano tanto las bondades como las carencias de los sistemas actuales de reserva y planificación de viajes.

Uno de los mayores inconvenientes que he observado en las agencias convencionales es la rigidez en la personalización y la falta de soporte inmediato e inteligente. Muchas plataformas presentan interfaces saturadas y poco intuitivas que dificultan la búsqueda de experiencias específicas, lo que a menudo genera errores en las reservas o una sensación de desatención que acaba perjudicando la confianza del viajero. Además, en un entorno digital cada vez más vulnerable, he detectado una falta de medidas de seguridad robustas en sitios de agencias medianas, donde la protección de los datos personales y de pago suele ser insuficiente.

Por ello, he decidido volcar mi entusiasmo por conocer el mundo en la creación de **Triptoyou**. Mi objetivo es desarrollar una plataforma web que no solo sea rápida y eficiente, sino que ofrezca una experiencia de usuario fluida, segura y enriquecida con herramientas, como la inteligencia artificial, para que planear un viaje sea algo sencillo y gratificante.

1.2 Objetivos del proyecto

El objetivo principal de este proyecto es consolidar y poner en práctica los conocimientos de desarrollo full-stack adquiridos durante este año en este curso, enfrentándome al reto de utilizar nuevas herramientas y en un entorno real.

Para alcanzar esta meta, me he propuesto los siguientes objetivos específicos:

- **Desarrollar una plataforma reactiva e intuitiva:** Utilizando **Angular**, crear una interfaz donde el catálogo de viajes sea dinámico y los filtros permitan encontrar el destino ideal con pocos clics, eliminando la complejidad de las agencias tradicionales.
- **Garantizar la máxima seguridad:** Implementar un sistema de **Autenticación de Doble Factor (2FA)** mediante códigos TOTP, asegurando que el acceso a la cuenta y la gestión de reservas cuenten con una capa de protección de grado profesional.
- **Integrar asistencia inteligente:** Implementar un **chatbot basado en la IA de Google Gemini**, que sea capaz de interactuar con los usuarios de forma natural, recomendando viajes del catálogo y resolviendo dudas en tiempo real basándose en el contexto del sitio.
- **Optimizar la gestión remota:** Gracias al uso de **Laravel** y una base de datos en la nube como **Neon.tech**, permitir que la administración del catálogo y las reservas sea accesible desde cualquier lugar, facilitando una gestión ágil y centralizada para los administradores de la agencia.
- **Enriquecer la información visual:** Integrar mapas dinámicos de **Google Maps** para que el usuario pueda localizar geográficamente sus destinos antes de realizar cualquier reserva.

- **Procesamiento de pagos reales:** Integrar una pasarela de pago segura y estandarizada en la industria (Stripe) para gestionar las transacciones de las reservas, permitiendo a la plataforma operar como un e-commerce completamente funcional.

2. Aspectos teóricos

2.1.1 ¿Qué es una SPA (Single Page Application)?

Una *Single Page Application* (SPA) o Aplicación de Página Única es una aplicación web que interactúa con el usuario reescribiendo dinámicamente la página actual con nuevos datos obtenidos del servidor, en lugar del método predeterminado del navegador de cargar páginas enteras desde cero. Esto proporciona una experiencia de usuario mucho más fluida y similar a la de una aplicación nativa.

En el contexto de una agencia de viajes digital como **Triptyou**, la función principal de esta arquitectura es facilitar la exploración fluida de destinos y el procesamiento de reservas. En la actualidad, este tipo de plataformas incluyen funcionalidades avanzadas como:

- **Catálogo interactivo y mapas:** Visualización dinámica de los viajes disponibles e integración con servicios de geolocalización (como Google Maps) sin recargas de página.
- **Seguridad y gestión de identidades:** Autenticación robusta mediante tokens y sistemas de Doble Factor de Autenticación (2FA) para proteger las cuentas y transacciones de los usuarios.
- **Asistencia inteligente:** Integración de Inteligencia Artificial (Chatbots) para ofrecer atención al cliente y recomendaciones de viajes en tiempo real.
- **Integración con bases de datos en la nube:** Sincronización continua a través de una API RESTful para garantizar la disponibilidad de los datos (precios, reservas, reseñas).

El uso de una arquitectura SPA optimiza la gestión del negocio turístico online, reduciendo los tiempos de espera del cliente, mejorando la interacción y aumentando la eficiencia operativa del sistema.

2.1.2 Roles de usuario en Triptyou

Para garantizar un correcto funcionamiento, seguridad y organización del sistema, la plataforma implementa distintos roles de usuario con permisos y accesos específicos:

Administrador:

- Gestiona el catálogo de viajes (creación, edición de precios, destinos y fechas).
- Administra las cuentas de los usuarios del sistema.
- Revisa y gestiona las reservas realizadas en la plataforma.
- Modera el contenido generado en el sistema.

Cliente (Usuario Registrado):

- Explora el catálogo de viajes, aplicando filtros de búsqueda dinámicos.
- Realiza y consulta sus reservas de viajes.

- Interactúa con el asistente virtual (Chatbot de IA) para resolver dudas o pedir recomendaciones.
- Publica reseñas y valoraciones (estrellas) sobre los viajes que ha realizado.

2.2 ¿Qué es Angular?

Angular es un framework de desarrollo de código abierto, mantenido principalmente por Google, diseñado para construir aplicaciones web de una sola página (SPA - Single Page Applications) eficientes y sofisticadas. Utiliza TypeScript como lenguaje principal, lo que aporta tipado estático y herramientas de desarrollo avanzadas sobre JavaScript estándar.

Principales características de Angular:

- **Arquitectura basada en componentes:** Permite dividir la interfaz de usuario en piezas reutilizables, independientes y fáciles de mantener.
- **Reactividad moderna:** En sus versiones más recientes, incorpora Signals, un sistema que optimiza el rendimiento actualizando la interfaz de forma granular solo cuando los datos cambian.
- **Enrutamiento del lado del cliente:** Permite navegar entre distintas vistas sin necesidad de recargar la página completa del navegador, mejorando drásticamente la velocidad de la aplicación.
- **Inyección de dependencias:** Facilita la gestión de servicios (como llamadas a la API o autenticación) haciéndolos disponibles en cualquier parte de la aplicación de manera eficiente.

2.2.1 Uso de Angular en Triptoyou

En este proyecto, Angular es la tecnología principal utilizada para el desarrollo de todo el frontend. Su uso permite:

- Crear un catálogo dinámico de viajes donde los filtros de búsqueda se aplican instantáneamente sin recargar la web.
- Gestionar la interfaz del Chatbot flotante y el sistema de reseñas de forma reactiva.
- Proteger rutas del cliente para asegurar que solo los usuarios identificados o con rol de administrador puedan acceder a paneles específicos.

2.3 ¿Qué es Laravel?

Laravel es un framework de código abierto para el desarrollo de aplicaciones y servicios web con PHP. Sigue el patrón de arquitectura Modelo-Vista-Controlador (MVC) y destaca por su sintaxis elegante y expresiva, diseñada para hacer que el proceso de desarrollo sea rápido, seguro y estructurado.

Principales características de Laravel:

- **ORM Eloquent:** permite interactuar con bases de datos utilizando sintaxis orientada a objetos en lugar de escribir sentencias SQL complejas manualmente.
- **Seguridad integrada:** Incluye protección nativa contra vulnerabilidades web comunes como inyecciones SQL.

- **Autenticación mediante API:** A través de paquetes como Laravel Sanctum, facilita la emisión y verificación de tokens para proteger APIs RESTful.
- **Ecosistema de paquetes:** Permite integrar fácilmente librerías de terceros mediante Composer (por ejemplo, para generar códigos QR o validar el doble factor de autenticación).

2.3.1 Uso de Laravel en Triptoyou

Laravel actúa como el backend de la aplicación, exponiendo una API REST a la que Angular se conecta. Se utiliza debido a sus múltiples ventajas:

- **Gestión de la lógica de negocio:** Procesa las reservas, calcula validaciones y gestiona los permisos de los usuarios y administradores.
- **Implementación de seguridad 2FA:** Utiliza librerías externas para generar las claves secretas TOTP y validarlas antes de permitir el acceso a áreas seguras.
- **Conector con la Inteligencia Artificial:** Es el encargado de enviar los prompts estructurados a la API de Gemini de forma segura, ocultando las claves privadas del lado del cliente.

2.4 ¿Qué es PostgreSQL y Neon.tech?

PostgreSQL es un sistema de gestión de bases de datos relacional de código abierto, reconocida por su potencia, flexibilidad y cumplimiento del estándar SQL. Neon.tech es una plataforma en la nube construida específicamente para alojar bases de datos PostgreSQL de forma escalable y sin necesidad de gestionar servidores físicos.

Principales características de PostgreSQL en Neon.tech:

- **Arquitectura Serverless:** Escala automáticamente los recursos según la demanda de la aplicación, optimizando los costes y el rendimiento.
- **Alta fiabilidad y estabilidad:** Garantiza la integridad de los datos.
- **Almacenamiento remoto seguro:** Elimina la necesidad de sincronizar bases de datos locales, centralizando la información en un entorno protegido.

2.4.1 Uso de PostgreSQL en Triptoyou

En este proyecto, PostgreSQL (alojado en Neon.tech) se utiliza como la base de datos principal y única del sistema, permitiendo:

- Almacenamiento centralizado y en tiempo real de información sobre viajes, reservas, reseñas y configuraciones de seguridad de los usuarios.
- Asegurar que las actualizaciones de precios o nuevos destinos realizados por el administrador se reflejen instantáneamente en las pantallas de todos los clientes conectados

2.5 ¿Qué es la Inteligencia Artificial Generativa (Google Gemini)?

Google Gemini es un modelo de lenguaje de última generación desarrollado por Google. Es un sistema de Inteligencia Artificial capaz de comprender, operar y combinar diferentes tipos de información, incluyendo texto, código e imágenes. Mediante su API pública, los desarrolladores pueden integrar este "cerebro" en sus propias aplicaciones.

Principales características de la API de Gemini:

- **Procesamiento de Lenguaje Natural:** Capacidad para entender preguntas complejas formuladas por humanos y generar respuestas coherentes y contextualizadas.
- **Configuración por "System Prompts":** Permite a los desarrolladores inyectar instrucciones estrictas ocultas para definir la personalidad, los límites y el conocimiento específico del modelo antes de que interactúe con el usuario.
- **Baja latencia:** Generación de respuestas rápidas ideales para entornos de mensajería instantánea o atención al cliente.

2.5.1 Uso de Gemini en Triptoyou

En este proyecto, se ha integrado la API de Gemini para dotar a la plataforma de un asistente virtual/Chatbot inteligente. Su función principal es:

- Brindar atención a la cliente automatizada las 24 horas del día.
- Responder dudas y recomendar destinos utilizando **exclusivamente** la información presente en la base de datos de la agencia (evitando "alucinaciones" de destinos no disponibles gracias a un System Prompt estructurado).
- Mejorar significativamente la experiencia de usuario, guiando a los clientes a través del catálogo de viajes mediante una conversación natural y amigable.

2.6 ¿Qué es DDEV y Docker?

Docker es una plataforma de código abierto que permite automatizar el despliegue de aplicaciones dentro de contenedores de software. Un contenedor empaqueta el código de la aplicación junto con todas sus dependencias, librerías y configuraciones, aislando el entorno para garantizar que funcione de manera correcta en cualquier infraestructura.

Por su parte, **DDEV** es una herramienta de código abierto construida sobre Docker, diseñada para facilitar la creación rápida de entornos de desarrollo. Su objetivo es simplificar la complejidad de configurar contenedores manualmente, ofreciendo entornos estandarizados para PHP, Node.js y bases de datos.

Principales características de DDEV:

- **Estandarización y aislamiento:** Garantiza que todos los desarrolladores que trabajen en el proyecto tengan exactamente las mismas versiones (PHP, Nginx, herramientas de consola).
- **Despliegue rápido:** Permite levantar y configurar un servidor web completo con un solo comando (ddev start), sin necesidad de instalar servidores locales pesados como XAMPP o WAMP.

- **Integración multiplataforma:** Funciona a la perfección en Linux, macOS y en Windows (yo use WSL2).
- **Herramientas integradas:** Incluye utilidades preconfiguradas esenciales para el desarrollo, como correo electrónico para pruebas (Mailpit) y acceso directo a administradores de bases de datos.

2.6.1 Uso de DDEV en Triptoyou

En este proyecto, se ha utilizado DDEV como la herramienta de orquestación principal para el entorno de desarrollo del Backend y el Frontend. Su uso ha aportado múltiples ventajas:

- **Entorno limpio en Windows:** Al utilizar DDEV sobre WSL2, no ha sido necesario instalar de forma nativa en el sistema operativo lenguajes como PHP o administradores de paquetes como Composer. Todo se ejecuta de forma aislada dentro de los contenedores de Docker.
- **Facilidad de portabilidad:** Gracias a la configuración automatizada en la carpeta. ddev, cualquier otra persona que clone el repositorio del proyecto podrá levantar la aplicación web completa en cuestión de minutos.

2.7 ¿Qué es vercel?

Vercel es una plataforma en la nube optimizada para el despliegue de frameworks de Frontend y sitios estáticos. Se ha elegido para alojar la aplicación cliente de Angular debido a su red de entrega de contenido global, que garantiza tiempos de carga mínimos, y su integración continua nativa con repositorios de GitHub

2.8 ¿Qué es Railway?

Railway es una plataforma de infraestructura en la nube que permite desplegar aplicaciones backend y bases de datos sin la complejidad de gestionar servidores tradicionales. Se ha seleccionado para alojar la API de Laravel por su capacidad de orquestación de contenedores, facilidad para inyectar variables de entorno seguras y ejecución continua de procesos

2.9 Estado del arte

En los últimos años, los sistemas de gestión y reserva para agencias de viajes han evolucionado drásticamente para adaptarse a un cliente cada vez más exigente. Tradicionalmente, las páginas web de turismo funcionaban como catálogos estáticos donde el usuario debía llamar por teléfono o enviar un correo para formalizar una reserva. Hoy en día, estas plataformas son ecosistemas dinámicos que ofrecen gran cantidad de funcionalidades: filtros de búsqueda en tiempo real, geolocalización, perfiles de usuario, reseñas y, más recientemente, asistencia automatizada mediante Inteligencia Artificial.

En el mercado actual, existen soluciones comerciales masivas como Booking, Expedia o Skyscanner, que integran potentes motores de búsqueda y algoritmos de recomendación. Sin embargo, estas gigantescas plataformas operan bajo modelos de negocio que imponen altas comisiones a los proveedores y agencias. Para una agencia de viajes pequeña o mediana, depender exclusivamente de estos intermediarios o intentar contratar un software a medida de esa magnitud suele resultar imposible de asumir debido a los costes de licencia, infraestructura y mantenimiento.

Hoy en día el desarrollo web ha experimentado un gran cambio. Se ha pasado de arquitecturas por páginas al desarrollo de Single Page Applications. Actualmente, el estándar de la industria es utilizar frameworks como React, Vue o Angular para el frontend, comunicándose a través de APIs RESTful con backends robustos desarrollados en Node.js, Python o PHP (Laravel). Esta separación quita la lógica de negocio de la interfaz, permitiendo una escalabilidad mucho mayor y una experiencia de usuario casi idéntica a la de una aplicación nativa.

Es así que, la combinación de **Angular** para la interfaz y **Laravel** para la API, representa una opción sólida, moderna y altamente demandada en el sector profesional. Angular aporta reactividad y estructuración gracias a TypeScript, mientras que Laravel asegura un procesamiento de datos rápido y seguro. Además, la tendencia ahora en infraestructura es migrar de bases de datos locales a soluciones en la nube, como **Neon.tech (PostgreSQL)**, lo que elimina los costes de mantenimiento de servidores físicos y garantiza la disponibilidad global de la información.

Por otro lado, la ciberseguridad y la atención al cliente marcan la diferencia actual de la innovación. Las plataformas modernas exigen medidas como la **Autenticación de Doble Factor** para proteger la información financiera y personal de los viajeros. Simultáneamente, el estándar de atención al cliente está cambiando de los clásicos menús de preguntas frecuentes hacia la integración de **Inteligencia Artificial Generativa**.

Entre las alternativas de código abierto y desarrollos académicos, existen múltiples proyectos en plataformas como GitHub que replican plataformas turísticas básicas. Sin embargo, la gran mayoría se limitan a operaciones CRUD (Crear, Leer, Actualizar, Borrar) convencionales. Muy pocos abordan una integración completa que sume mapas interactivos, seguridad criptográfica (TOTP) y, sobre todo, un Chatbot impulsado por un modelo de lenguaje avanzado como **Google Gemini**, capaz de entender el contexto del catálogo y asesorar al usuario en tiempo real.

Este trabajo de fin de grado trata de mejorar el estándar, proponiendo una solución adaptada a una agencia de viajes contemporánea. La intención no es competir en tamaño con multinacionales como Booking, sino ofrecer una plataforma completa, segura y tecnológicamente avanzada que demuestre cómo una agencia mediana puede aprovechar herramientas gratuitas, de código abierto y de inteligencia artificial para ofrecer un servicio digital de máxima calidad.

3. Planificación

Para la planificación del proyecto he decidido organizar el trabajo mediante jornadas de aproximadamente 2 horas día sí, día no, ajustando esta dedicación dependiendo de la complejidad de las tareas por hacer, los bloqueos técnicos encontrados y el tiempo restante de cada sprint.

3.1 Riesgos posibles

En el desarrollo de una plataforma web completa intervienen múltiples tecnologías, lo que conlleva ciertos riesgos inherentes:

- **Problemas personales:** Enfermedad, accidente o cambios en el entorno laboral/académico que repercutan en el número de horas disponibles para dedicar al desarrollo del proyecto.
- **Errores en la estimación de fechas:** Provocados por la falta de un horario estricto y fijo para el proyecto o por la inexperiencia en la integración de algunas de las herramientas que abarca el ecosistema web moderno.
- **Desconocimiento o curva de aprendizaje de herramientas:** Retrasos derivados del tiempo necesario para dominar el uso de Angular, la implementación del 2FA (algoritmos TOTP) en Laravel, o la sintaxis específica para la API de Google Gemini.
- **Problemas técnicos de entorno y dependencias:** Conflictos de compatibilidad entre los sistemas operativos (por ejemplo, ejecución de comandos Node/npm de Windows sobre subsistemas Linux/WSL) o fallos en los contenedores de Docker (DDEV) que impidan levantar el servidor local.
- **Fallas en la integración de servicios externos:** Caídas temporales o cambios en las políticas de la API de Google Maps, Google Gemini o el servicio de base de datos en la nube Neon.tech.

3.2 Actuaciones previstas ante los riesgos

- **Problemas personales:** Como el equipo de desarrollo está compuesto únicamente por el alumno, la planificación de fechas y horas dedicadas al proyecto se verá afectada de forma directa. Se procurará compensar en fines de semana o días con mayor disponibilidad.
- **Errores en la estimación de fechas:** Se documentará adecuadamente el error de la estimación en el tablero Kanban y se replanificarán o dividirán las tareas más grandes en subtareas más pequeñas si es conveniente.
- **Desconocimiento y problemas de entorno técnico:** Para mitigar problemas de dependencias, se ha estandarizado el uso de herramientas aisladas (como DDEV para el backend y Node Version Manager en el frontend de Linux).
- **Falta de viabilidad en el proyecto:** Esta situación debe evitarse a toda costa. Para ello, se ha realizado un estudio previo de la documentación oficial de Angular, Laravel y Gemini, asegurando que todas las piezas son compatibles entre sí.

Si se produce la aparición de alguno de los problemas citados anteriormente o de otros que no se hayan contemplado, dadas las características del proyecto, podría producirse un impacto en la duración de este. Ante cualquier problema crítico, nos veremos obligados a realizar un esfuerzo extra por aumentar el número de horas diarias dedicadas al desarrollo del proyecto para cumplir con los plazos de entrega.

3.3 Presupuesto

Dado que para la arquitectura y despliegue del proyecto he utilizado herramientas de código abierto (Angular, Laravel, Docker/DDEV) y plataformas que ofrecen capas gratuitas para desarrollo (alojamiento de la base de datos en Neon.tech, API de mapas y uso de la API de Google Gemini). Además, el equipo de desarrollo está compuesto solamente por el alumno y todo el trabajo de programación se va a realizar en el equipo informático del propio alumno. Por lo tanto, este proyecto no tiene ningún coste económico directo.

3.4 Entregables

La finalización del Trabajo de Fin de Grado comprende la entrega de los siguientes elementos:

- **Memoria del proyecto:** Documentación exhaustiva detallando el análisis, diseño, implementación y pruebas del sistema.
- **Producto final:** La plataforma completa de "Triptoyou", que se compone de:
 - Aplicación web *Frontend*.
 - Aplicación *Backend*.
 - Base de datos en la nube completamente estructurada y operativa.
- **Listado de pruebas de integración:** Casos de prueba detallando el correcto funcionamiento de las características críticas (reservas, 2FA, Chatbot).
- **Manual de despliegue:** manual que indica como hacer el despliegue de la web
- **Manual de administrador:** enseña a usar la página web

3.5 Metodologías ágiles

Para este proyecto he decidido utilizar metodologías ágiles, concretamente un marco de trabajo adaptado de Scrum y Kanban. Estoy utilizando sprints con un tiempo estimado de dos semanas, en los cuales agrupo y ejecuto las tareas propuestas.

Para gestionar los Sprints y el Backlog del producto, estoy utilizando una tabla Kanban en **GitHub Projects**, que me permite organizar y visualizar el flujo de trabajo en tiempo real. Este tipo de tablero divide las tareas en columnas según su estado, como "Por hacer" (To Do), "En progreso" (In Progress) y "Hecho" (Done). Esta metodología facilita enormemente el seguimiento del avance, la identificación temprana de cuellos de botella y la gestión de prioridades de manera clara y visual.

3.5.1 Sprint 1

Fecha: 4 al 18 de marzo Para mi primer sprint me he propuesto configurar la arquitectura base del proyecto: establecer el entorno de desarrollo aislado con contenedores (DDEV), crear y conectar la base de datos PostgreSQL alojada en la nube (Neon.tech) e implementar la API en Laravel para el registro y el login de usuarios y administradores utilizando tokens (Sanctum). También he comenzado a redactar la memoria del proyecto.

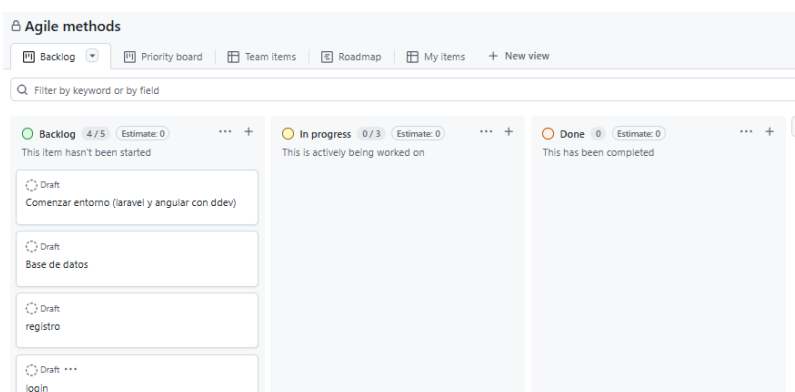


Figura 1 Tablero Kanban Sprint 1

3.5.2 Sprint 2

Fecha: 18 de marzo al 1 de abril Este es el aspecto de mi tablero Kanban al finalizar mi primer sprint. Para el segundo sprint he propuesto desarrollar toda la funcionalidad del panel de administrador (operaciones CRUD para gestionar viajes y usuarios en el backend), iniciar el desarrollo del frontend interactivo utilizando Angular (componentes, servicios y enrutamiento básico) y continuar documentando.

He conseguido finalizar en el tiempo esperado el sistema de autenticación, la estructura en Neon.tech y la base visual del catálogo de viajes en el frontend.

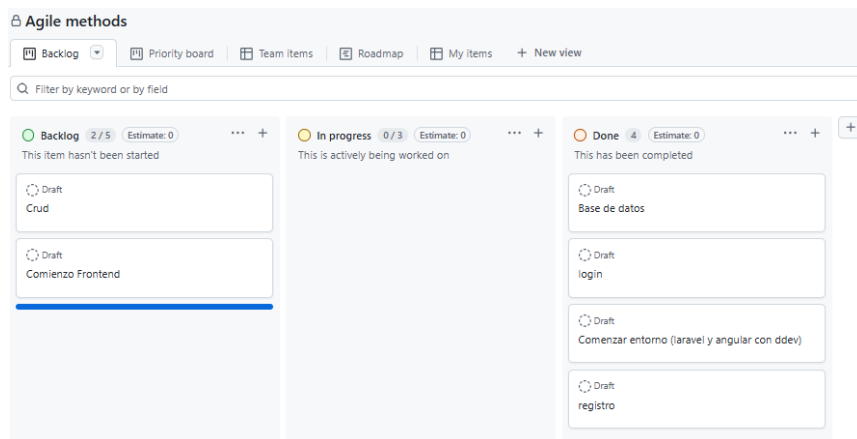


Figura 2 Tablero Kanban Sprint 2

3.5.3 Sprint 3

Fecha: 1 al 15 de abril Este es el aspecto de mi tablero Kanban tras mi segundo sprint. Para el tercer sprint me he propuesto implementar la vista de detalles de cada viaje, integrar los mapas dinámicos de Google Maps mediante SafePipe en Angular para la geolocalización de los destinos, y desarrollar el sistema interactivo de reseñas con valoración por estrellas.

He conseguido finalizar la funcionalidad del menú de administrador, la conexión HTTP entre el catálogo de Angular y la API de Laravel, y la visualización correcta de los mapas integrados.

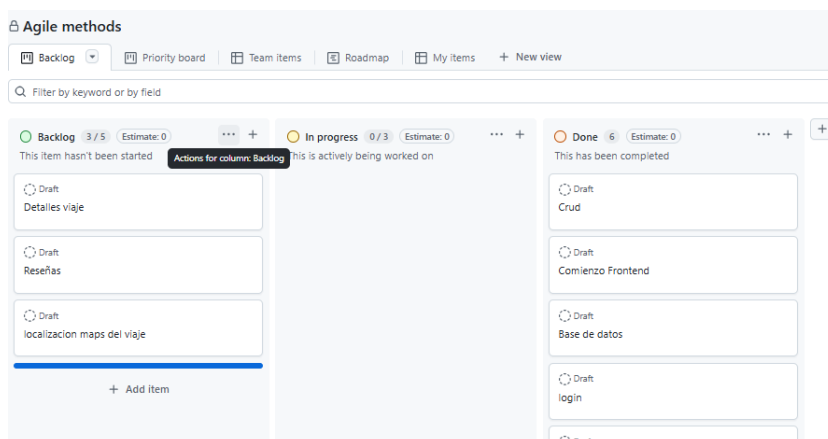


Figura 3 Tablero Kanban Sprint 3

3.5.4 Sprint 4

Fecha: 15 al 29 de abril Para este sprint me he propuesto implementar los filtros de búsqueda dinámicos en el catálogo, diseñar el área de "Perfil de usuario" y añadir una capa de ciberseguridad avanzada implementando la Autenticación de Doble Factor (2FA) con generación de códigos QR. Sigo realizando la documentación del código.

He conseguido finalizar todo el apartado de las reseñas (envío y validación en base de datos), la lectura dinámica de variables.

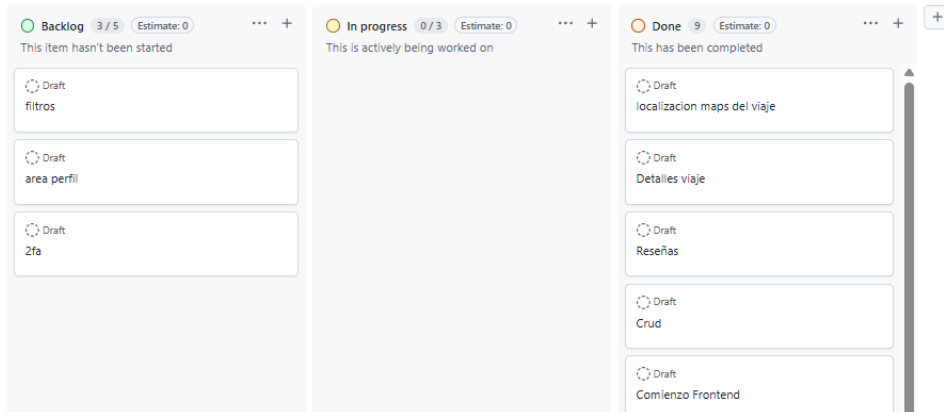


Figura 4 Tablero Kanban Sprint 4

3.5.5 Sprint 5

Fecha: 29 de abril al 10 de mayo Este es el aspecto de mi tablero Kanban encarando la recta final. Para este último sprint me he propuesto integrar el Chatbot de Inteligencia Artificial utilizando la API de Google Gemini. Además, me he centrado en depurar bugs importantes del frontend (como el problema de retención del scroll al cambiar de página y las fugas de memoria en las animaciones del componente principal), reservas y finalizar por completo la memoria con sus correspondientes manuales.

He conseguido implementar el 2FA con éxito en la plataforma, el asistente virtual es completamente funcional y se nutre del catálogo real, y el rendimiento de la aplicación ha sido optimizado.

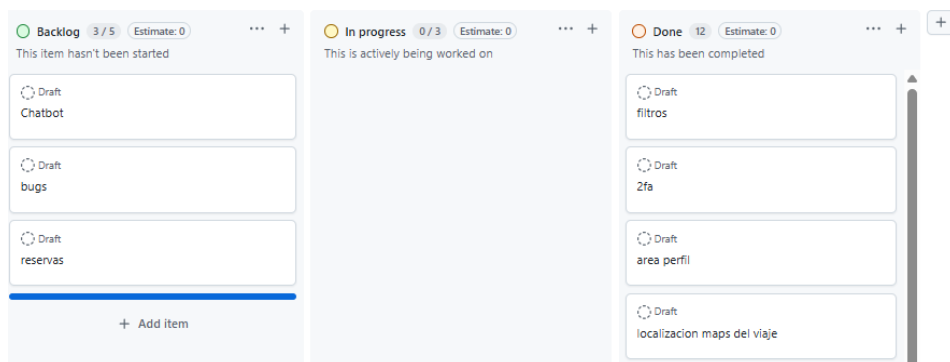


Figura 5 Tablero Kanban Sprint 5

4. Análisis

4.1 Tipos de usuarios

Para asegurar una correcta separación de responsabilidades y proteger las áreas sensibles del sistema, la plataforma **Triptoyou** distingue entre tres perfiles principales de interacción:

- **Visitante (Usuario no autenticado):** Es cualquier persona que accede a la página web sin haber iniciado sesión. Tiene acceso público al catálogo de viajes, puede utilizar los filtros de búsqueda y ver la información detallada de los destinos (incluyendo el mapa de Google Maps y las reseñas de otros usuarios). Sin embargo, si desea realizar una reserva o dejar una valoración, el sistema le redirigirá a la pantalla de registro/login.
- **Cliente (Usuario Registrado):** Es el núcleo de la plataforma. Para acceder, debe autenticarse mediante su correo y contraseña (y opcionalmente, superar la verificación de Doble Factor 2FA si la ha activado).
 - Puede gestionar su perfil y activar/desactivar la seguridad 2FA mediante códigos TOTP.
 - Tiene permisos para reservar viajes, consultar su historial de reservas y cancelarlas si es necesario.
 - Puede interactuar libremente con el Chatbot (Gemini) para solicitar asistencia o recomendaciones.
 - Puede publicar reseñas y valoraciones de 1 a 5 estrellas en los viajes.
- **Administrador:** Es el encargado del mantenimiento operativo del negocio. Inicia sesión a través del mismo sistema de autenticación, pero su cuenta posee privilegios elevados.
 - Desde su panel de control, puede gestionar el catálogo: crear nuevos viajes, editar precios, descripciones, fechas y ubicaciones, o eliminar ofertas obsoletas.
 - Administra a los usuarios del sistema, pudiendo visualizar la base de datos de clientes o intervenir en caso de incidencias.
 - Revisa y gestiona las reservas realizadas en la plataforma.

4.2 Requisitos funcionales

A continuación, se detallan los requisitos funcionales del sistema, organizados por módulos lógicos, incluyendo sus posibles flujos alternativos o de error.

Código	Nombre del requisito	Descripción y flujos
R1	Gestión de Identidad y Seguridad	Control de acceso y protección de cuentas

R1.1	Registro de nuevo usuario	R1.1.1 Usuario registrado con éxito y sesión iniciada. R1.1.2 Error de validación (ej. email ya existe o contraseña débil). R1.1.3 Error de conexión con la BD en Neon.tech.
R1.2	Inicio de sesión (Login)	R1.2.1 Autenticación correcta mediante Laravel Sanctum. R1.2.2 Credenciales inválidas.
R1.3	Configuración de 2FA	R1.3.1 Generación correcta de código QR y clave secreta. R1.3.2 Activación exitosa tras validación del código TOTP. R1.3.3 Código TOTP inválido o caducado. R1.3.4 Desactivación del 2FA por parte del usuario.
R1.4	Inicio de sesión con 2FA	R1.4.1 Redirección a pantalla de verificación 2FA tras login. R1.4.2 Acceso concedido tras introducir pin de 6 dígitos. R1.4.3 Bloqueo temporal por múltiples intentos fallidos.
R2	Gestión del Catálogo de Viajes	Exploración e información turística
R2.1	Visualización del catálogo	R2.1.1 Obtención exitosa del listado de viajes desde la API. R2.1.2 Lista vacía si no hay viajes disponibles.
R2.2	Filtrado dinámico	R2.2.1 Filtrado correcto por precio, destino o fechas en tiempo real mediante Angular. R2.2.2 Ningún resultado coincide con los filtros.
R2.3	Detalle y Geolocalización	R2.3.1 Carga exitosa de información detallada de un viaje.

		R2.3.2 Carga correcta del Iframe de Google Maps mediante SafePipe.
R2.4	CRUD de Viajes (Admin)	R2.4.1 Creación, edición o eliminación de un viaje con éxito. R2.4.2 Error de validación al intentar guardar datos incompletos.
R3	Sistema de Reservas	Contratación de servicios
R3.1	Realizar reserva	R3.1.1 Reserva generada y vinculada al usuario autenticado. R3.1.2 Fallo por falta de disponibilidad o error del servidor.
R3.2	Historial de reservas	R3.2.1 El cliente visualiza correctamente sus reservas pasadas y futuras.
R3.3	Procesamiento de Pago	R3.3.1 Pago completado con éxito mediante tarjeta bancaria (Stripe). R3.3.2 Pago denegado (fondos insuficientes o tarjeta inválida).
R3.4	Confirmación de Reserva	R3.4.1 La reserva solo cambia a estado "Confirmado" tras recibir la respuesta positiva del <i>Webhook</i> de Stripe.
R5	Asistencia por IA (Chatbot)	Atención al cliente 24/7
R5.1	Inicializar Chatbot	R5.1.1 Despliegue correcto de la ventana flotante en la interfaz.

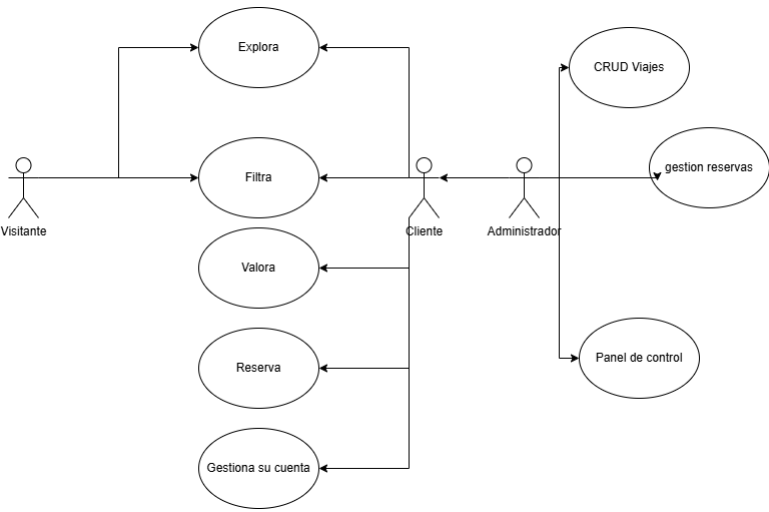
R5.2	Interacción con Gemini	R5.2.1 El usuario envía un prompt y recibe una respuesta estructurada (viñetas, negritas) basada en el catálogo real. R5.2.2 El bot se niega a responder temas fuera de contexto (mecanismo anti-alucinaciones).
------	------------------------	--

4.3 Identificación de casos de uso

El sistema está diseñado para dar soporte a las interacciones de los distintos actores previamente definidos. Las acciones principales que delimitan las fronteras del sistema son:

Actores:

- **Visitante:** Explora la web, utiliza filtros de búsqueda y visualiza mapas y reseñas.
- **Cliente:** Hereda las funciones del Visitante y suma la gestión de seguridad de su cuenta (2FA), la realización de reservas, la publicación de reseñas y la consulta al Chatbot de inteligencia artificial.
- **Administrador:** Hereda las funciones del Cliente y suma el control total sobre el inventario (CRUD de viajes), moderación del sistema y gestión del panel de usuarios.



5 Diseño de la base de datos

Para el desarrollo de **Triptoyou**, se ha optado por una arquitectura de base de datos centralizada y nativa en la nube, prescindiendo de bases de datos locales. Esta decisión técnica se justifica por la naturaleza web del proyecto (Single Page Application), donde los clientes y administradores acceden desde distintos dispositivos a través de un navegador y necesitan interactuar con datos.

El motor de base de datos elegido es **PostgreSQL**, un sistema de gestión de bases de datos relacional de código abierto conocido por su robustez e integridad referencial. Para su despliegue y alojamiento, se está utilizando el servicio de **Neon.tech**, lo que garantiza alta

disponibilidad, copias de seguridad automatizadas y escalabilidad sin necesidad de administrar la infraestructura del servidor.

Todas las interacciones con esta base de datos se realizan de manera segura a través de la API construida en **Laravel**, haciendo uso de Eloquent, el cual previene inyecciones SQL y facilita la gestión de las relaciones entre tablas.

5.1 Esquema Relacional en PostgreSQL (Neon.tech)

El modelo de datos relacional se ha diseñado en tercera forma normal (3NF) para evitar la redundancia de datos. El sistema pivota sobre cuatro tablas principales fuertemente relacionadas entre sí:

- **Tabla users (Usuarios):** Almacena la información de identidad, roles y, de manera crucial para este proyecto, los parámetros criptográficos para la seguridad avanzada. Contiene las credenciales cifradas y los campos `two_factor_secret` y `two_factor_enabled` que permiten el funcionamiento del 2FA (Google Authenticator).
- **Tabla trips (Catálogo de Viajes):** Es la entidad central del negocio. Guarda toda la información de los destinos ofrecidos, incluyendo los títulos, descripciones, precios, fechas de disponibilidad y el campo `location`, que almacena las coordenadas o direcciones exactas requeridas para renderizar los mapas interactivos a través de la API de Google Maps en el frontend.
- **Tabla bookings (Reservas):** Actúa como tabla transaccional. Registra las operaciones de compra uniendo a un usuario con un viaje en una fecha determinada. Contiene claves foráneas apuntando a `users` y `trips`, garantizando la trazabilidad de las ventas.
- **Tabla reviews (Reseñas y Valoraciones):** Permite la interacción social en la plataforma. Relaciona a un cliente con un viaje que haya realizado, almacenando un comentario de texto y una puntuación numérica (rating de 1 a 5 estrellas).

5.2 Tablas

Para detallar la estructura interna de las entidades más críticas del sistema, a continuación, se expone el diccionario de datos de las tablas principales:

Tabla: users

Campo	Tipo de Dato	Restricciones	Descripción
id	BIGINT	PK, Auto-incremental	Identificador único del usuario.
name	VARCHAR(255)	NOT NULL	Nombre completo o de visualización.

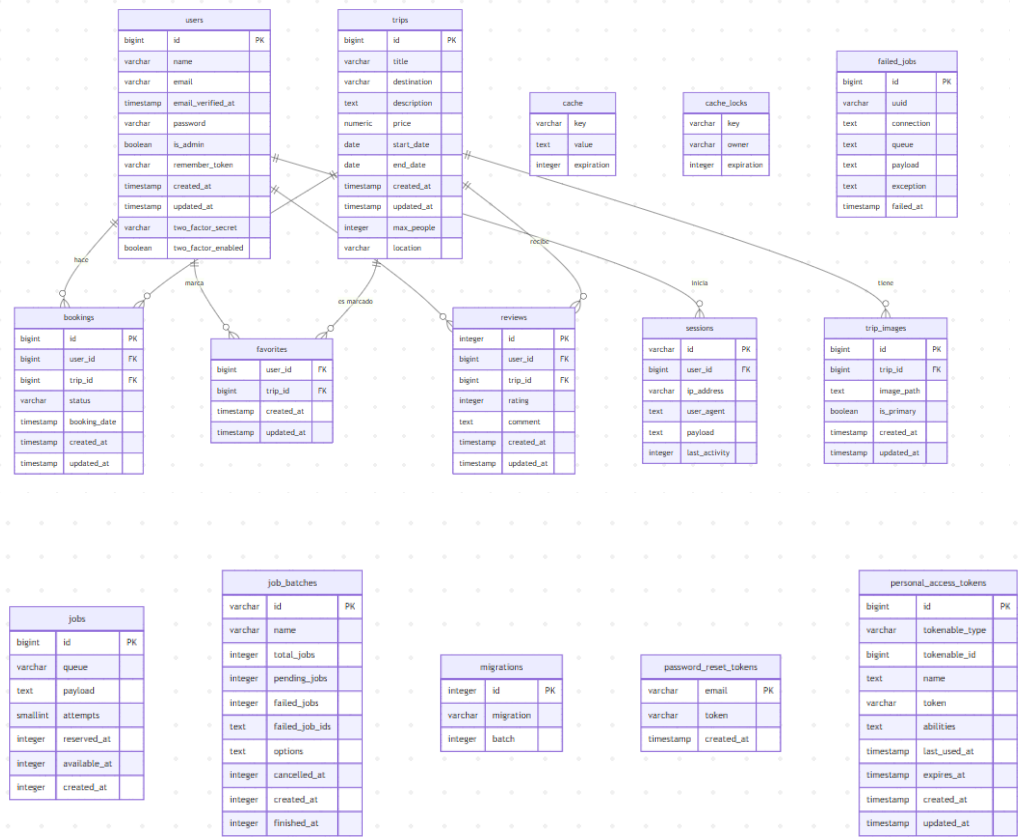
email	VARCHAR(255)	UNIQUE, NOT NULL	Correo electrónico (credencial de acceso).
password	VARCHAR(255)	NOT NULL	Contraseña hasheada mediante Bcrypt.
two_factor_secret	VARCHAR(255)	NULLABLE	Clave secreta cifrada para el algoritmo TOTP (2FA).
two_factor_enabled	BOOLEAN	DEFAULT FALSE	Bandera que indica si el 2FA está activo.
role	VARCHAR(50)	DEFAULT 'client'	Define los permisos (client, admin).

Tabla: trips

Campo	Tipo de Dato	Restricciones	Descripción
id	BIGINT	PK, Auto-incremental	Identificador único del viaje.
destination	VARCHAR(255)	NOT NULL	Ciudad o país de destino.
location	TEXT	NULLABLE	Cadena de búsqueda para el Iframe de Google Maps.
price	DECIMAL(10,2)	NOT NULL	Coste del viaje en euros.
start_date	DATE	NOT NULL	Fecha de inicio del viaje.
end_date	DATE	NOT NULL	Fecha de finalización.

Tabla: reviews

Campo	Tipo de Dato	Restricciones	Descripción
id	BIGINT	PK, Auto-incremental	Identificador único de la reseña.
user_id	BIGINT	FK (users.id)	Usuario que publica la valoración.
trip_id	BIGINT	FK (trips.id)	Viaje que está siendo valorado.
rating	SMALLINT	CHECK (1-5)	Valoración en formato de estrellas.
comment	TEXT	NULLABLE	Comentario cualitativo del usuario.



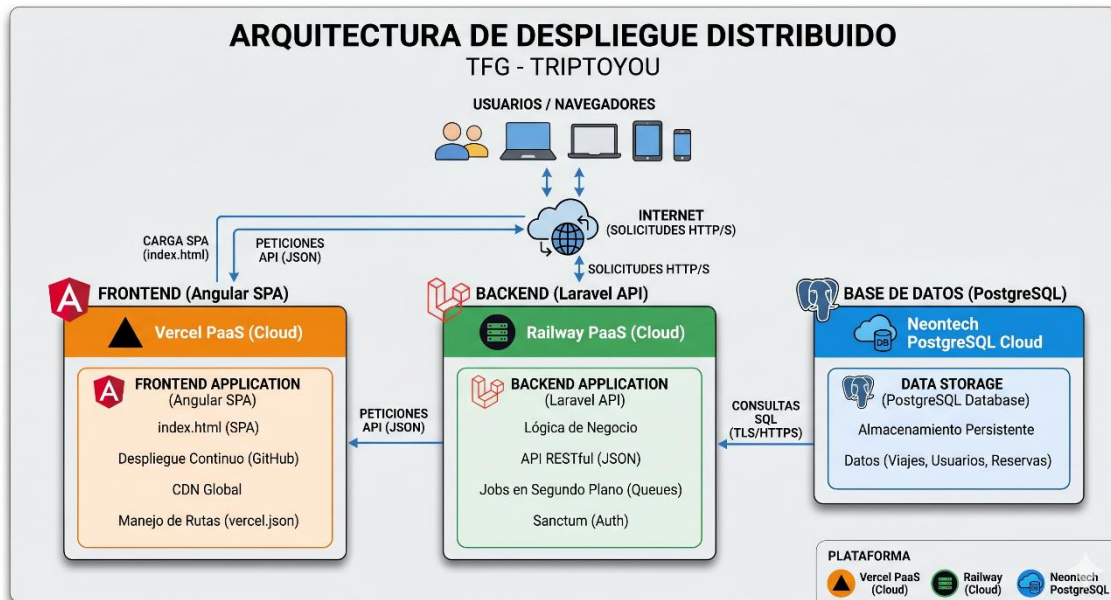
5.3 Arquitectura de despliegue

El sistema ha sido diseñado bajo una arquitectura de microservicios distribuidos en la nube, separando físicamente el cliente, el servidor y los datos para garantizar la escalabilidad y alta disponibilidad:

Frontend (Cliente): Alojado en Vercel, sirviendo la Single Page Application (SPA) desarrollada en Angular.

Backend (API REST): Desplegado en Railway, actuando como el motor lógico del sistema.

Capa de Datos: Alojada en Neon.tech, garantizando el almacenamiento persistente mediante PostgreSQL. Esta separación (Serverless para el cliente y PaaS para el servidor) asegura que, si el tráfico de usuarios aumenta drásticamente, cada capa puede escalar sus recursos de forma independiente.



6. Implementación del sistema

La implementación de la plataforma web **Triptoyou** se ha desarrollado utilizando un enfoque full-stack moderno, separando el Frontend y el Backend para crear una arquitectura cliente-servidor robusta y escalable.

El desarrollo se ha apoyado en herramientas de contenedores como **Docker** y **DDEV** para el entorno local. Esto garantiza que la configuración del servidor web, las versiones de PHP y las dependencias sean idénticas independientemente del sistema operativo o equipo en el que se desarrolle, eliminando los clásicos problemas de "funciona en mi máquina".

6.1 Arquitectura del sistema

A diferencia de la arquitectura tradicional en tres capas de las aplicaciones de escritorio de varias páginas, Triptoyou emplea un modelo de **Single Page Application que usa una API RESTful**.

6.1.1 El Frontend (Cliente) - Angular

La capa de presentación ha sido desarrollada íntegramente con **Angular (v18)** y **TypeScript**. Esta capa se ejecuta directamente en el navegador del usuario y es responsable de la interfaz de usuario y la experiencia de usuario.

- **Componentes Standalone:** La aplicación está dividida en componentes independientes (ej. TripCardComponent, ChatbotComponent, ReviewFormComponent), lo que favorece la reutilización del código.
- **Reactividad con Signals:** Se ha implementado el nuevo sistema de reactividad de Angular basado en Signals. Esto permite que la interfaz se actualice de forma granular y eficiente cuando los datos cambian (por ejemplo, al aplicar un filtro de precio)
- **Servicios e Interceptores:** La comunicación con el servidor se centraliza a través de servicios inyectables (ej. ApiService). Además, se utilizan interceptores HTTP para adjuntar automáticamente el token de autorización en cada petición que requiera seguridad.

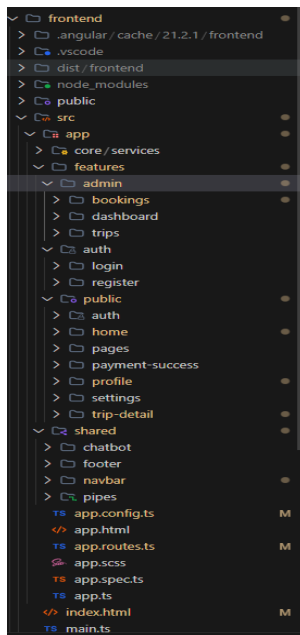


Figura 6 Estructura del Frontend en Angular

6.1.2 El Backend (Servidor) - Laravel

La lógica de negocio, la validación de reglas de seguridad y la comunicación con la base de datos residen en el servidor, desarrollado con **Laravel (PHP)**. El backend no genera código HTML (vistas); su única función es recibir peticiones HTTP, procesarlas y devolver respuestas en formato JSON (API REST).

- **Controladores (Controllers):** Actúan como intermediarios. Reciben las peticiones del frontend (por ejemplo, una solicitud para crear una reseña), validan los datos de entrada y llaman a los modelos correspondientes.
- **Modelos (Eloquent ORM):** Representan las tablas de la base de datos como objetos de PHP (ej. Trip, User, Review). Eloquent facilita las consultas complejas y gestiona las relaciones (un usuario tiene muchas reservas).
- **Rutas (API Routes):** El archivo routes/api.php define los endpoints o puntos de acceso (ej. GET /api/trips, POST /api/reviews).
- **Middleware de Seguridad:** Se utilizan middlewares como auth: sanctum para proteger las rutas, asegurando que solo los usuarios autenticados puedan ejecutar ciertas acciones.

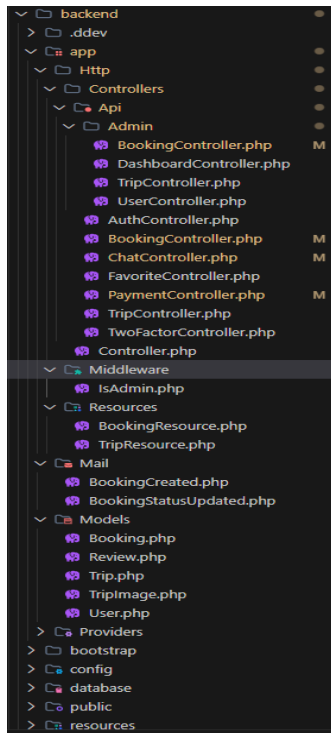


Figura 7 Estructura del Backend en Laravel

6.2 Comunicación y Gestión de Datos

La comunicación entre el cliente (Angular) y el servidor (Laravel) se realiza exclusivamente mediante peticiones asíncronas HTTP utilizando el formato estándar **JSON** para el intercambio de información.

6.2.1 Conexión a PostgreSQL (Neon.tech)

La persistencia de datos se gestiona a través de **PostgreSQL** alojado en la plataforma **Neon.tech**.

La conexión entre Laravel y Neon.tech se configura de manera segura a través del archivo de variables de entorno (.env), evitando exponer credenciales en el código fuente. Las variables esenciales configuran el host, el puerto, la base de datos, el usuario y la contraseña proporcionados por Neon.

Ejemplo de configuración de conexión (ocultando datos sensibles):

Fragmento de código

```
DB_CONNECTION=pgsql
DB_HOST=ep-curly-water-a2xyz123.eu-central-1.aws.neon.tech
DB_PORT=5432
DB_DATABASE=triptoyou_db
DB_USERNAME=admin_user
DB_PASSWORD=*****
DB_SSLMODE=require
```

Al utilizar Eloquent ORM de Laravel, todas las consultas a la base de datos se ejecutan internamente utilizando sentencias parametrizadas. Esto protege estructuralmente a la aplicación frente a ataques de **Inyección SQL (SQLi)**.

6.2.2 Ejemplo de Flujo de Datos (Obtener Viajes)

Para ilustrar la separación de capas, este es el flujo de ejecución cuando un usuario entra en la página principal para ver el catálogo:

1. **Angular (Frontend):** El componente HomeComponent se inicializa y llama al método `getTrips ()` del `TripService`.
2. **Petición HTTP:** Angular envía una petición GET a la URL <https://api.triptoyou.local/api/trips>.
3. **Laravel (Backend):** El enrutador recibe la petición y la dirige al `TripController@index`.
4. **Eloquent (ORM):** El controlador ejecuta `Trip::all ()` (o aplica filtros si los hay). Laravel traduce esto a una consulta SQL segura y la envía a Neon.tech.
5. **PostgreSQL (Neon.tech):** Ejecuta la consulta y devuelve los registros.
6. **Respuesta JSON:** Laravel formatea los datos y los envía de vuelta a Angular con un código HTTP 200 (OK).
7. **Renderizado:** Angular recibe el JSON, el HTML se repinta mostrando las tarjetas de los viajes al usuario.

6.3 Gestión de Viajes, Reseñas y Usuarios

El sistema permite a los administradores gestionar el catálogo de viajes los usuarios a través de paneles de control dedicados en el Frontend de Angular. Las operaciones CRUD (Crear, Leer, Actualizar y Eliminar) se comunican mediante peticiones HTTP hacia la API de Laravel.

A diferencia de sistemas antiguos que dependían de sincronizaciones manuales o bases de datos locales, **Triptoyou funciona en tiempo real**. Si un administrador actualiza el precio de un destino o lo marca como "Agotado", la base de datos PostgreSQL en Neon.tech se actualiza instantáneamente, y cualquier cliente navegando por el catálogo verá reflejado el cambio de inmediato, evitando conflictos de reservas o errores de datos. Además, se ha integrado un sistema de reseñas, donde los usuarios pueden dejar valoraciones (1 a 5 estrellas) de forma dinámica, y cuyos promedios se recalculan en el backend para mostrarse en el catálogo principal.

6.4 Sistema de Reservas y Confirmación Digital

Durante el proceso de compra, el usuario selecciona el viaje, hace el pago y confirma la reserva. Esta acción genera un registro en la tabla `bookings` del servidor.

En lugar de imprimir tickets físicos, el sistema está preparado para generar comprobantes digitales. La API de Laravel asocia la reserva al ID del usuario mediante el token de sesión, garantizando que un usuario no pueda manipular la petición para reservar en nombre de otro. Toda esta información queda accesible de forma instantánea en el panel de "Mis Reservas" del cliente, donde puede consultar el estado y los detalles de su viaje. Aparte, se envía un correo al usuario.

6.5 Analíticas, Moderación y Panel de Administración

Los administradores pueden:

- Visualizar un resumen en tiempo real del número total de reservas activas y el volumen de negocio generado.
- Controlar el comportamiento de los usuarios en la plataforma.
- Moderar el sistema de reseñas: si un usuario publica un comentario inapropiado, el administrador tiene la capacidad de eliminarlo de la base de datos central. Toda esta información persiste en PostgreSQL, asegurando su disponibilidad remota para futuras auditorías o integraciones.

6.6 Pasarela de pago: Integración con Stripe

La implementación se ha dividido en dos fases arquitectónicas:

1. **Frontend (Angular):** Se ha integrado el Stripe en la pantalla de pago (Checkout). Esta librería incluye un formulario de tarjeta de crédito directamente en la aplicación de Angular. Sin embargo, por motivos de seguridad, los números de tarjeta nunca tocan nuestro servidor; se envían directamente desde el navegador del cliente a los servidores de Stripe, los cuales nos devuelven un Token seguro.
2. **Backend (Laravel):** Utilizando Stripe para PHP (stripe/stripe-php), el servidor de Laravel recibe el *Token* del cliente y la información de la reserva. A continuación, Laravel ejecuta el cargo (Charge) contra la API de Stripe .

Además, para garantizar la consistencia de los datos frente a cortes de conexión (por ejemplo, si el cliente cierra el navegador justo al pagar), se han implementado **Webhooks**. Esto significa que Stripe envía una notificación asíncrona de servidor a servidor (Laravel) confirmando que el pago ha sido exitoso, y es en ese momento exacto cuando la base de datos de PostgreSQL actualiza la reserva al estado 'Pagado'."

6.7 Ciberseguridad Avanzada y Autenticación de Doble Factor (2FA)

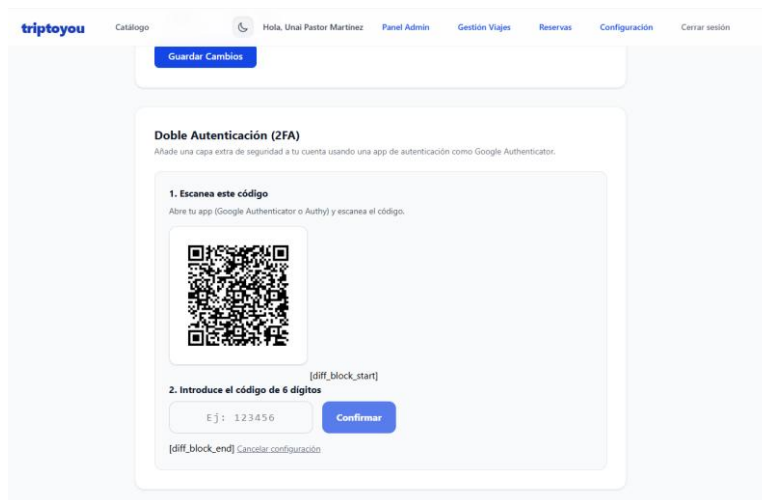
La seguridad es uno de los pilares fundamentales de Triptoyou. El mecanismo tradicional de login ha sido sustituido por un sistema de autenticación robusto basado en **Laravel Sanctum**, el cual emite y valida Tokens para cada sesión de usuario. Las contraseñas jamás se guardan en texto plano; son procesadas mediante algoritmos de hashing.

Para proteger las cuentas frente a accesos no autorizados (como filtraciones de credenciales), se ha implementado un sistema de **Autenticación de Doble Factor (2FA) basado en TOTP** (Time-based One-Time Password).

El flujo de seguridad funciona de la siguiente manera:

1. El backend de Laravel genera una clave secreta única para el usuario y crea un código QR.
2. Angular renderiza este QR de forma segura para que el cliente lo escanee con aplicaciones como Google Authenticator.

3. En cada inicio de sesión posterior, tras introducir su contraseña, la API requerirá el código temporal de 6 dígitos. Solo si la validación criptográfica en el servidor es exitosa, se emitirá el token de acceso final.



6.8 Entorno de Desarrollo y Portabilidad (Docker y DDEV)

El despliegue de las aplicaciones web exige contenerización para garantizar que funcionen de manera idéntica en cualquier servidor (Linux, macOS, o Windows a través de WSL2 como es mi caso).

Para este proyecto se ha utilizado **DDEV**, una herramienta de entorno de desarrollo local basada en **Docker**. DDEV orquesta los contenedores necesarios de forma automática:

- Un contenedor web para el servidor Nginx y PHP (ejecutando Laravel).
- Interceptores de correo local para pruebas.
- Herramientas de terminal integradas (Node.js) para ejecutar el servidor de desarrollo de Angular.

Esta arquitectura garantiza que la aplicación sea 100% portable. Cualquier desarrollador que clone el repositorio desde GitHub solo tiene que ejecutar `ddev start` para tener todo el ecosistema (Frontend y Backend) funcionando en cuestión de segundos, sin tener que instalar lenguajes o servidores web en su máquina anfitriona de forma manual.

6.9 Dependencias y Librerías Externas (Ecosistema)

En un proyecto Full-Stack, las extensiones se gestionan mediante administradores de paquetes formales. Se ha hecho uso de **Composer** para el backend (PHP) y **NPM** para el frontend (Node/Angular). Entre las dependencias más destacadas se encuentran:

- **Para el Backend (Laravel/Composer):**
 - `pragmarx/google2fa`: Librería criptográfica esencial para la generación de claves y validación del algoritmo TOTP del Doble Factor.
 - `bacon/bacon-qr-code`: Encargado de convertir las cadenas de autenticación en imágenes vectoriales SVG o PNG para los códigos QR del 2FA.

- Cliente HTTP (Guzzle) nativo de Laravel: Utilizado para la comunicación segura mediante peticiones POST con la **API externa de Google Gemini** (Chatbot).
- **Para el Frontend (Angular/NPM):**
 - Tailwind CSS: Un framework de utilidades CSS utility-first que permite diseñar toda la interfaz web de manera ágil, responsiva y altamente personalizable sin abandonar los archivos HTML.

```

{
  "name": "frontend",
  "version": "0.0.0",
  "scripts": {
    "start": "ng serve",
    "build": "ng build",
    "watch": "ng build --watch --configuration development",
    "test": "ng test"
  },
  "private": true,
  "packageManager": "npm@11.8.0",
  "dependencies": {
    "@angular/common": "^21.2.0",
    "@angular/compiler": "^21.2.0",
    "@angular/core": "^21.2.0",
    "@angular/forms": "^21.2.0",
    "@angular/platform-browser": "^21.2.0",
    "@angular/router": "^21.2.0",
    "@tailwindcss/postcss": "^4.2.1",
    "rxjs": "~7.8.0",
    "tslib": "^2.3.0"
  },
  "devDependencies": {
    "@angular/build": "^21.2.1",
    "@angular/cli": "^21.2.1",
    "@angular/compiler-cli": "^21.2.0",
    "autoprefixer": "^10.4.27",
    "jsdom": "^28.0.0",
    "postcss": "^8.5.8",
    "prettier": "^3.8.1",
    "tailwindcss": "^4.2.1",
    "typescript": "~5.9.2",
    "vitest": "^4.0.8"
  }
}

```

Figura 8 Package.json

```

{
  "$schema": "https://getcomposer.org/schema.json",
  "name": "laravel/laravel",
  "type": "project",
  "description": "The skeleton application for the Laravel framework.",
  "keywords": ["laravel", "framework"],
  "license": "MIT",
  "require": {
    "php": "^8.2",
    "bacon/bacon-qrcode": "^3.1",
    "laravel/framework": "^12.0",
    "laravel/sanctum": "^4.0",
    "laravel/tinker": "^2.10.1",
    "pragmarx/google2fa": "^9.0",
    "stripe/stripe-php": "^19.4"
  },
  "require-dev": {
    "fakerphp/faker": "^1.23",
    "laravel/pail": "^1.2.2",
    "laravel/pint": "^1.24",
    "laravel/sail": "^1.41",
    "mockery/mockery": "^1.6",
    "nunomaduro/collision": "^8.6",
    "phpunit/phpunit": "^11.5.3"
  },
  "autoload": {
    "psr-4": {
      "App\\": "app/",
      "Database\\Factories\\": "database/factories/",
      "Database\\Seeders\\": "database/seeders/"
    }
  },
  "autoload-dev": {
    "psr-4": {
      "Tests\\": "tests/"
    }
  },
  "scripts": {
    "setup": [
      "composer install",
      "@php -r '\\file_exists('.env') || copy('.env.example', '.env')\\'",
      "@php artisan key:generate",
      "@php artisan migrate --force",
      "npm install"
    ],
    "scripts": {
      "dev": {
        "test": [
          "@php artisan config:clear --ansi",
          "@php artisan test"
        ],
        "post-autoload-dump": [
          "Illuminate\\Foundation\\ComposerScripts::postAutoloadDump",
          "@php artisan package:discover --ansi"
        ],
        "post-update-cmd": [
          "@php artisan vendor:publish --tag=laravel-assets --ansi --force"
        ],
        "post-root-package-install": [
          "@php -r '\\file_exists('.env') || copy('.env.example', '.env')\\'"
        ],
        "post-create-project-cmd": [
          "@php artisan key:generate --ansi",
          "@php -r '\\file_exists(database/database.sqlite) || touch(database/database.sqlite)\\'",
          "@php artisan migrate --graceful --ansi"
        ],
        "pre-package-uninstall": [
          "Illuminate\\Foundation\\ComposerScripts::prePackageUninstall"
        ]
      },
      "extra": {
        "laravel": {
          "dont-discover": []
        }
      },
      "config": {
        "optimize-autoloader": true,
        "preferred-install": "dist",
        "sort-packages": true,
        "allow-plugins": {
          "pestphp/pest-plugin": true,
          "php-http/discovery": true
        }
      },
      "minimum-stability": "stable",
      "prefer-stable": true
    }
  }
}

```

Figura 9 Composer.json

Figura 10 Composer.json

6.10 Rendimiento y Optimización Web

El rendimiento en una SPA (Single Page Application) se mide analizando la carga en el navegador del cliente y los tiempos de respuesta del servidor.

La aplicación es extremadamente eficiente gracias a:

1. **Compilación AOT (Ahead-of-Time) de Angular:** El código de Angular se compila en archivos JavaScript altamente optimizados antes de enviarse al navegador. Al usar los nuevos Standalone Components, se carga exclusivamente el código necesario para cada vista.
2. **Reactividad por Signals:** Angular no tiene que redibujar toda la página al recibir nuevos datos (como la respuesta del Chatbot o el filtrado de un viaje), sino que actualiza únicamente el nodo exacto del DOM.
3. **Latencia de Neon.tech:** La conexión a la base de datos en la nube demuestra tiempos de respuesta adecuado para consultas complejas, permitiendo que el catálogo y el 2FA respondan de forma comprensiblemente rápida para ser una base de datos gratuita.
4. **Optimización Multimedia: Conversión a WebP:** El manejo de imágenes de alta calidad es habitualmente el mayor cuello de botella en los tiempos de carga de una plataforma turística. Para optimizar el rendimiento, he implementado un sistema de conversión automática en el Backend (Laravel). Cuando un administrador sube una nueva fotografía al catálogo, el servidor lo convierte dinámicamente al formato **WebP** antes de añadirlo en la base de datos.

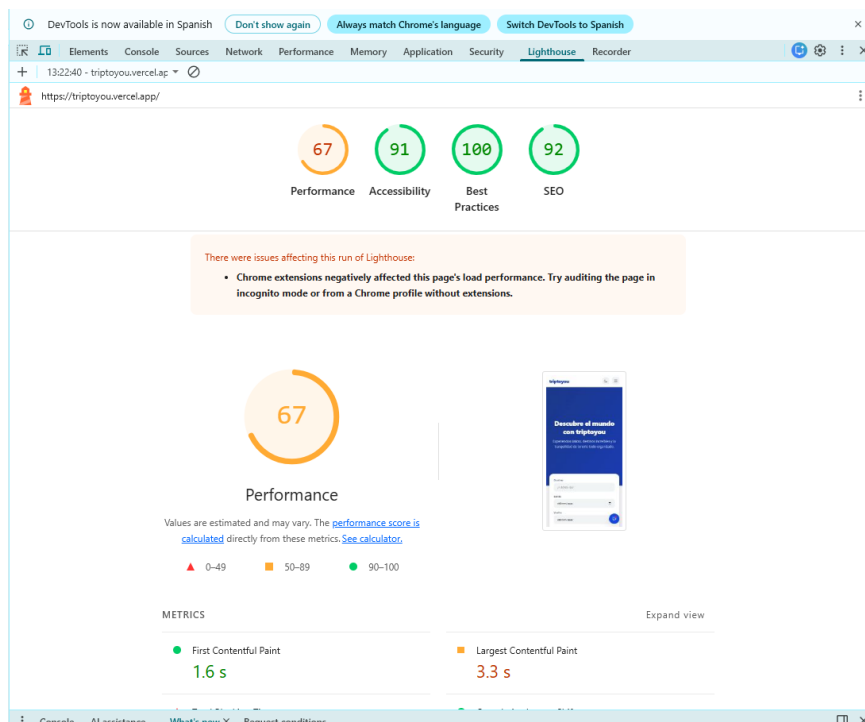


Figura 11 Captura lighthouse

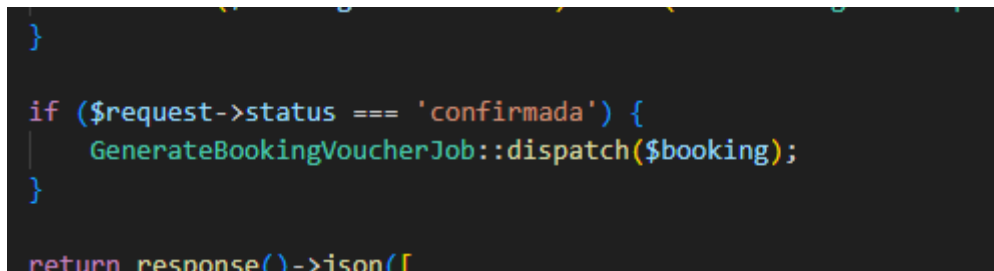
6.11 Problemas encontrados y soluciones técnicas

Durante el desarrollo e integración de los distintos módulos de la plataforma web, surgieron varios desafíos técnicos relacionados con el rendimiento y la experiencia de usuario. En el desarrollo web las peticiones síncronas de larga duración pueden bloquear la respuesta del servidor o inutilizar el navegador del cliente.

1. Latencia en la generación de comprobantes y notificaciones (El problema del hilo principal)

Al inicio, cuando un usuario hacía una reserva, el sistema de Laravel (Backend) procesaba la inserción en la base de datos, generaba el comprobante digital (PDF) y preparaba el envío de la confirmación. Al realizarse de forma síncrona, el servidor no devolvía la respuesta a Angular hasta terminar todas estas acciones, dejando al usuario frente a una pantalla de carga durante varios segundos.

Para solucionar esto, he implementado el uso de **Colas y Trabajos en segundo plano (Queues & Jobs)** en Laravel, lo cual es el equivalente arquitectónico a delegar tareas en hilos (Threads/Tasks) en aplicaciones nativas.

A screenshot of a code editor with a dark background. It shows a PHP code snippet from a Laravel controller. The code is as follows:

```
}  
  
if ($request->status === 'confirmada') {  
    GenerateBookingVoucherJob::dispatch($booking);  
}  
  
return response()->json([
```

Figura 12 Uso de Jobs en Laravel para procesos asíncronos

He decidido utilizar este sistema de Jobs porque es mucho más eficiente. En lugar de obligar al proceso principal de la petición HTTP a esperar la generación del documento, la tarea se asigna a un "trabajador" en segundo plano. Esto permite que el controlador devuelva un código HTTP 200 casi instantáneamente, mejorando la percepción de velocidad por parte del usuario.

2. Asincronía y tiempos de respuesta con la IA (Google Gemini)

Otro problema similar se presentó al integrar el Chatbot. Las consultas a la API de Google Gemini requieren un tiempo de procesamiento natural por parte del modelo de lenguaje (habitualmente entre 2 y 4 segundos). Si la interfaz de Angular se bloqueaba esperando la respuesta, la experiencia resultaba nefasta.

Para solucionarlo, en el Frontend he hecho uso exhaustivo de la asincronía de JavaScript. Al enviar el prompt, el sistema cambia inmediatamente un estado a `isLoading = true`, lo que renderiza una animación de "Escribiendo..." en el chat. Una vez la request se resuelve y los datos llegan desde Laravel, el estado se actualiza reactivamente. Esto permite que el usuario pueda seguir haciendo scroll en el catálogo o interactuando con la web mientras la Inteligencia Artificial procesa su respuesta en segundo plano.

3. Comportamiento de la posición del scroll al navegar

Durante el desarrollo del Frontend, detecté un problema de uso relacionado con la naturaleza de la arquitectura Single Page Application. En las páginas web tradicionales, al hacer clic en un enlace, el navegador recarga la página por completo y te sitúa en la parte superior. Sin embargo, en Angular, al cambiar de vista dinámicamente sin recargar la página, la aplicación retenía la posición del scroll.

El problema práctico era que, si un cliente hacía scroll hacia abajo para ver los últimos viajes del catálogo y luego hacía clic para entrar en su perfil, la nueva página de perfil se cargaba desplazada hacia abajo, obligando al usuario a subir manualmente para ver el principio del contenido.

Para solucionar este comportamiento y ofrecer una navegación natural, modifiqué la configuración global del enrutador de Angular. Se añadió la propiedad de restauración de scroll, la cual fuerza automáticamente a la interfaz a reiniciar la ventana a la parte superior de forma imperceptible cada vez que se detecta un cambio de ruta, mejorando la experiencia del cliente.

7. Pruebas de integración

En esta sección se desarrollarán pruebas manuales, conforme la funcionalidad del sistema se vaya desarrollando. Son muy importantes dentro del desarrollo de la aplicación web, ya que la superación de dichas pruebas dará paso al desarrollo del resto de módulos y permite comprobar si la API de Laravel y el *Frontend* de Angular siguen cumpliendo con los requisitos descritos inicialmente.

A continuación, se proponen los casos de prueba que se deberán desarrollar a medida que se va dotando de funcionalidad al sistema.

7.1 Gestión de usuarios y Seguridad (2FA)

Registrar Usuario

Caso de Prueba: CP1.1.1	
Entrada	Resultado esperado
Registrar un usuario con un email no existente en la aplicación	El sistema procesa la petición en Laravel, añade el usuario a PostgreSQL y se inicia sesión correctamente en Angular.
Resultado obtenido	Prueba correcta

Caso de Prueba: CP1.1.2	
Entrada	Resultado esperado
Registrar un usuario con un email ya existente en la base de datos	El sistema no crea el usuario y el <i>frontend</i> muestra un mensaje de error indicando que el correo ya está en uso.
Resultado obtenido	Prueba correcta

Autenticación de Doble Factor (2FA)

Caso de Prueba: CP1.2.1	
Entrada	Resultado esperado
El usuario solicita activar el 2FA en su perfil	El sistema genera una clave secreta y devuelve un código QR visible en pantalla para escanear con Google Authenticator.
Resultado obtenido	Prueba correcta
Caso de Prueba: CP1.2.2	
Entrada	Resultado esperado
Iniciar sesión con una cuenta que tiene el 2FA activado introduciendo un código TOTP incorrecto	El servidor deniega el acceso (Error 401/422) y se muestra un mensaje de "Código de verificación inválido".

Resultado obtenido	Prueba correcta
---------------------------	-----------------

7.2 Gestión del Catálogo de Viajes

Creación de Viajes (Administrador)

Caso de Prueba: CP2.1.1	
Entrada	Resultado esperado
El administrador registra un nuevo viaje con todos los datos correctos (destino, precio, fechas, location)	El sistema guarda el viaje y este aparece inmediatamente disponible en el catálogo público.
Resultado obtenido	Prueba correcta
Caso de Prueba: CP2.1.2	
Entrada	Resultado esperado
Registrar un viaje con campos obligatorios vacíos (ej. sin precio)	Laravel rechaza la petición por error de validación y Angular resalta el campo vacío en rojo indicando el error.
Resultado obtenido	Prueba correcta

Exploración y Mapas (Cliente)

Caso de Prueba: CP2.2.1	
Entrada	Resultado esperado

El usuario aplica un filtro de búsqueda por "Destino" o "Precio" en el catálogo	El listado de viajes se actualiza dinámicamente en pantalla sin recargar la página web gracias a Angular <i>Signals</i> .
Resultado obtenido	Prueba correcta
Caso de Prueba: CP2.2.2	
Entrada	Resultado esperado
El usuario accede a los detalles de un viaje que tiene una ubicación registrada	El sistema carga correctamente el Iframe de Google Maps mostrando la geolocalización del destino.
Resultado obtenido	Prueba correcta

7.3 Gestión de Reservas

Realizar Reserva

Caso de Prueba: CP3.1.1	
Entrada	Resultado esperado
Un usuario autenticado pulsa el botón "Reservar" en un viaje disponible	La API registra la reserva vinculando el user_id con el trip_id y redirige al usuario a su panel de "Mis Reservas" con un mensaje de éxito.
Resultado obtenido	Prueba correcta
Caso de Prueba: CP3.1.2	
Entrada	Resultado esperado

Un usuario Visitante (no autenticado) intenta hacer una reserva	El sistema bloquea la acción mediante un Guard de Angular y lo redirige automáticamente a la pantalla de Login.
Resultado obtenido	Prueba correcta

Cancelar/Eliminar Reserva

Caso de Prueba: CP3.2.1	
Entrada	Resultado esperado
El usuario (o administrador) cancela una reserva desde el panel	El sistema elimina o cambia el estado de la reserva y esta desaparece del listado activo.
Resultado obtenido	Prueba correcta

Procesamiento de Pagos

Caso de Prueba: CP3.3.1	
Entrada	Resultado esperado
El usuario completa el proceso de reserva introduciendo la tarjeta de prueba "4242 4242..." proporcionada por Stripe.	El pago se procesa correctamente, la reserva cambia a estado "Pagado" en PostgreSQL y el usuario ve la pantalla de confirmación.
Resultado obtenido	Prueba correcta

7.4 Sistema de Reseñas

Caso de Prueba: CP4.1.1	
--------------------------------	--

Entrada	Resultado esperado
Un cliente envía una reseña con 5 estrellas y un comentario válido	La reseña se guarda en la base de datos y la media de valoración del viaje se recalcula y se actualiza visualmente.
Resultado obtenido	Prueba correcta

7.5 Asistencia por Inteligencia Artificial (Chatbot Gemini)

Interacción Contextual

Caso de Prueba: CP5.1.1	
Entrada	Resultado esperado
El usuario pregunta al Chatbot por recomendaciones de viajes a "Japón" (destino existente en BD)	El modelo de Gemini procesa el catálogo inyectado y devuelve una respuesta estructurada recomendando el viaje y su precio.
Resultado obtenido	Prueba correcta
Caso de Prueba: CP5.1.2	
Entrada	Resultado esperado
El usuario pregunta al Chatbot por temas ajenos a la agencia (ej. "dame una receta de cocina") o destinos no disponibles	El Chatbot obedece su <i>System Prompt</i> , se niega a responder sobre temas externos y redirige la conversación al catálogo de la agencia.
Resultado obtenido	Prueba correcta

8. Manuales

He elaborado dos guías orientadas a los desarrolladores y a los empleados

- **Manual de Despliegue:** Orientado a desarrolladores. Explica cómo clonar el repositorio desde GitHub, cómo levantar los contenedores de Docker utilizando **DDEV** (ddev start), la configuración del archivo .env para conectar con la base de datos en **Neon.tech** y los comandos de Node (npm install y ng serve) para levantar el entorno de Angular.
- **Manual de Administrador:** Orientado a los empleados que usan la web. Explica como usar la página web

9. Conclusión y ampliaciones

A lo largo del desarrollo de este proyecto, he conseguido implementar con éxito **Triptoyou**, una plataforma web moderna orientada a cubrir las necesidades reales de una agencia de viajes. El sistema desarrollado destaca por su fluidez, su arquitectura, su capacidad para operar 100% en la nube y su integración Inteligencia Artificial (Google Gemini). Estas características no solo modernizan la experiencia del usuario final, sino que también garantizan la alta disponibilidad, escalabilidad y seguridad de los datos.

El estar familiarizado con tecnologías como **Angular, TypeScript, Laravel y PostgreSQL**, me ha permitido llevar a cabo un desarrollo ágil y con excelentes resultados en cuanto a rendimiento y hacerlo mantenible en el tiempo. La implementación de funcionalidades complejas como la Autenticación de Doble Factor (2FA) mediante algoritmos TOTP, la gestión dinámica del catálogo con mapas interactivos y la integración de un asistente virtual inteligente, son logros que reflejan un trabajo y web efectiva.

Durante el proceso también surgieron retos importantes, el problema con el scroll, nuevas características o los bloqueos de la interfaz provocados por la latencia en las respuestas de la IA. Estos problemas fueron resueltos gracias al uso avanzado de asincronía y la delegación de procesos pesados en colas de trabajo en el servidor. Estas soluciones mejoraron considerablemente la usabilidad del sistema, permitiendo una experiencia inmersiva y sin interrupciones.

Posibles ampliaciones

- **Aplicación Móvil Nativa:** Desarrollar una aplicación móvil utilizando frameworks multiplataforma (como Ionic o Flutter) que consuma la misma API de Laravel ya construida.
- **Generación de Itinerarios con IA:** Evolucionar el Chatbot actual para que, además de responder dudas, sea capaz de generar y enviar por correo electrónico itinerarios de viaje en PDF completamente personalizados según los gustos del cliente.
- **Internacionalización (i18n):** Implementar soporte multi-idioma para que la plataforma sea accesible a turistas internacionales, traduciendo tanto la interfaz como el contenido del catálogo de forma dinámica.

Con estas posibles mejoras, **Triptoyou** podría consolidarse como una solución completa y altamente competitiva, adaptándose a cualquier contexto y necesidad dentro del sector turístico global.

10. Bibliografía

- **Documentación Oficial y Frameworks**
 - Angular Team. (2024). *Angular Documentation: Signals and Standalone Components*. Google. Recuperado de <https://angular.dev/>
 - Otwell, T. (2024). *Laravel 11.x - The PHP Framework for Web Artisans*. Laravel LLC. Recuperado de <https://laravel.com/docs/11.x>
 - Neon Database. (2024). *Neon Serverless PostgreSQL Documentation*. Recuperado de <https://neon.tech/docs/introduction>
 - Tailwind Labs. (2024). *Tailwind CSS - Rapidly build modern websites without ever leaving your HTML*. Recuperado de <https://tailwindcss.com/docs>
- **Inteligencia Artificial y Machine Learning**
 - Google. (2024). *Gemini API Documentation & Prompt Engineering Guidelines*. Google AI for Developers. Recuperado de <https://ai.google.dev/docs>
 - Zhao, W. X., et al. (2023). *A Survey of Large Language Models*. arXiv preprint arXiv:2303.18223. Recuperado de <https://arxiv.org/abs/2303.18223>
- **Seguridad y Arquitectura de Software**
 - M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). *RFC 6238: TOTP: Time-Based One-Time Password Algorithm*. Internet Engineering Task Force (IETF). Recuperado de <https://datatracker.ietf.org/doc/html/rfc6238>
 - Stauffer, M. (2019). *Laravel: Up & Running: A Framework for Building Modern PHP Apps* (2nd ed.). O'Reilly Media.
 - Freeman, A. (2022). *Pro Angular 16* (5th ed.). Apress.
 - Pragmarx. (2023). *Google2FA - A Google Authenticator compatible Two-Factor Authentication package for PHP*. GitHub. Recuperado de <https://github.com/antonioribeiro/google2fa>
- **Herramientas de Desarrollo y Entorno**
 - DDEV Foundation. (2024). *DDEV Docs - Local Web Development Environment*. Recuperado de <https://ddev.com/>
 - GitHub. (2024). *Build and ship software on a single, collaborative platform*. Recuperado de <https://github.com/>
 - Draw.io. (2024). *Diagramming and charting tool*. Recuperado de <https://www.drawio.com/>